LIN - ALG one-stop

Gradient operator

wrt a matrix

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1d}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{n1}} & \cdots & \frac{\partial f}{\partial A_{nd}} \end{bmatrix}$$

where $f : \mathbb{R}^{n \times d} \to \mathbb{R}$

wrt a vector

$$\nabla_\theta f(\theta) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \vdots \\ \frac{\partial f}{\partial \theta_d} \end{bmatrix}$$

where $f : \mathbb{R}^{d \times 1} \to \mathbb{R}$

VERY USE FUL!

$$\nabla_x (b^T x) = b$$

$$\nabla_x (x^T A x) = 2Ax \quad (\text{when } A = A^T)$$

$$\nabla_x^2 (x^T A x) = A \quad (\text{when } A = A^T)$$

Hessian Operator:

$$H \in \mathbb{R}^{d \times d} = \begin{bmatrix} \frac{\partial^2}{\partial \theta_1 \theta_1} & \cdots \\ \vdots & \ddots \\ & \frac{\partial^2}{\partial \theta_i \theta_j} \end{bmatrix} \iff H_{ij} = \frac{\partial^2}{\partial \theta_i \partial \theta_j}$$

$$\Rightarrow \frac{\partial}{\partial x_k} \frac{1}{2} x^T A x = \frac{\partial}{\partial x_k} \frac{1}{2} \sum_{i=1, i \neq k}^{n} A_{ik} x_i x_k + \frac{\partial}{\partial x_k} \frac{1}{2} \sum_{j=1, j \neq k}^{n} A_{kj} x_k x_j + \frac{\partial}{\partial x_k} \frac{1}{2} A_{kk} x_k x_k$$

Example of component-wise linalg.

PD / PSD

$A \in \mathbb{R}^{n \times n}$ is PSD ($A \succeq 0$) if $A = A^T$ & $x^T A x \geq 0$ for all $x \in \mathbb{R}^n$

$A \in \mathbb{R}^{n \times n}$ is PD ($A \succ 0$) if $A = A^T$ & $x^T A x > 0$ for all $x \in \mathbb{R}^n$

Rank & Nullspace

$\text{Null}(A) \triangleq \{ x \mid A\vec{x} = \vec{0} \}$

$\text{Rank}(A) \triangleq$ # of linearly independent Rows of A.

Rank-Nullity Theorem: $\text{Rank}(A) + \text{Nullity}(A) = n$ for $A \in \mathbb{R}^{m \times n}$ (n = # of cols of A)

$A \in \mathbb{R}^{m \times n}$ & full-rank → $m \geq n$ (tall matrix) → $\text{null}(A) = 0$

full-rank → $m < n$ (wide matrix) → $\text{null}(A) = n - m$

Spectral Thm

If $A \in \mathbb{R}^{n \times n}$ is diagonalizeable

$A = T \Lambda T^{-1}$ where Λ is a diagonal matrix $\begin{bmatrix} \lambda_1 & \cdots & 0 \\ \vdots & \ddots & \\ 0 & & \lambda_n \end{bmatrix}$

& $\lambda^{(i)}$, $\vec{t}^{(i)}$ are eigenvalue/eigenvector pairs of A!

Probability one-stop

Density	Moment Generating Function	Mixing parameter
Normal	$\exp(\mu t + \sigma^2 t^2/2)$	μ, σ^2
Gamma	$[\lambda/(\lambda - t)]^\rho$	ρ
Poisson $P(X=k) = \frac{e^{-\lambda}\lambda^k}{k!}$	$\exp[\lambda(e^t - 1)]$	λ
Binomial	$(q + pe^t)^n$ $P_x = \binom{n}{x} p^x q^{n-x}$	n
Negative Binomial	$[p/(1 - qe^t)]^n$	n
Logistic	$e^{at}\Gamma(1 - \beta t)\Gamma(1 + \beta t)$	a
Extreme value	$e^{at}\Gamma(1 - \beta t)$	a
Double exponential	$e^{at}/[1 - (\beta t)^2)]$	a
Generalized Poisson	$\exp[\lambda(h(t) - 1)]$	λ
Generalized Binomial	$(q + ph(t))^n$	n
Generalized Neg. Binomial	$[p/(1 - qh(t))]^n$	n

→ easy

→ continuous analog of Binomial

→ "# of expected heads in n coin flips" (AKA a series of Bernoulli distr.)

BAYE'S RULE: $P(y|x) = \frac{P(x|y)\,P(y)}{P(x)} = \frac{P(x|y)\,P(y)}{\sum_{\text{all } i} P(x|y=i)\,P(y=i)}$

MULTIVARIATE GAUSSIAN: $\mathcal{N}(M \in \mathbb{R}^{n\times 1}, \Sigma \in \mathbb{R}^{n\times n})$

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2}\,|\Sigma|^{1/2}} \exp\left(-\tfrac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)\right)$$

$|\Sigma|^{1/2} \hookrightarrow \det(\Sigma)$

$$\text{Cov}(X_{\mathbb{R}^{n\times 1}}) = E[(X - E[X])(X - E[X])^T] = \Sigma_{\mathbb{R}^{n\times n}}$$
$$= E[ZZ^T] - (E[Z])(E[Z])^T = \Sigma_{\mathbb{R}^{n\times n}}$$

} Two definitions for the same variable

Σ → always symm. & PSD

Intuition on covariance Matrix Σ

$$\Sigma = \begin{bmatrix} a & b \\ b & a \end{bmatrix}$$

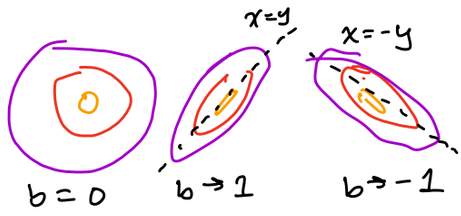
values of a do not matter as they are physically insignificant (saying $\text{Cov}(x, x) = 1$ is not shocking)

$b \triangleq \text{Cov}(X, Y) = \text{Cov}(Y, X)$

$b = 0$ x & y are independent

$b = 1$ x & y are pos. correlated

$b = -1$ x & y are neg. correlated



EXPECTATION, VARIANCE, COVARIANCE, CORR

$\text{Var}(X) = E[(X - E[X])]^2 = E[X^2] - E[X]^2$

$\text{Var}(X_1 + X_2) = \text{Var}(X_1) + \text{Var}(X_2) + 2\,\text{COV}(X_1 + X_2)$

$\text{Cov}(X_1, X_2) = E[(X_1 - E[X_1])(X_2 - E[X_2])] = E[XY] - E[X]E[Y]$

$\text{Corr}(X_1, X_2) = \frac{\text{COV}(X_1, X_2)}{\sqrt{\text{VAR}(X_1)\,\text{VAR}(X_2)}}$

LEMMA 0.1 (from PS 0)

let $X \sim \mathcal{N}(\mu, \Sigma)$ $A \in \mathbb{R}^{m \times n}$ $b \in \mathbb{R}^m$

then $AX + b \sim \mathcal{N}(A\mu + b, A\Sigma A^T)$

$A\mu + b$: linearity of expectation; $A\Sigma A^T$: derived!

INDEPENDANCE:

$P(AB) = P(A)P(B)$

$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(A)P(B)}{P(B)} = P(A)$

PROPERTIES OF EXPECTATION

$E[c] = c$

let $E[X^2] = \mu^2 + \sigma^2$!

$E[cf(x)] = c\,E[f(x)]$

$E[\sum_i f_i(x)] = \sum_i E[f_i(x)]$

$E[E[X|Y]] = E[X]$ // Law of Total Expectation → Tricky!

PROPERTIES OF VARIANCE

$\text{Var}(c) = c$

$\text{Var}(cf(x)) = c^2\,\text{Var}(f(x))$

PROPERTIES of COVARIANCE

If $X \perp Y$, then $E[XY] = E[X]E[Y] \rightarrow \text{Cov}(X, Y) = 0$ for $X \perp Y$

$\text{Cov}(X, X) = E[XX] - E[X]E[X] = \text{Var}[X]$

CHAIN RULE

$f(x_1, x_2, \ldots, x_n) = f(x_1)\,f(x_2|x_1)\ldots f(x_n|x_1, x_2 \ldots x_{n-1})$

$= f(x_1) \sum_{i=2}^{n} f(x_i | x_1, \ldots x_{i-1})$

Example Distributions

Distribution	PDF or PMF	Mean	Variance
Bernoulli(p)	$\begin{cases} p, & \text{if } x = 1 \\ 1-p, & \text{if } x = 0. \end{cases}$	p	$p(1-p)$
Binomial(n, p)	$\binom{n}{k} p^k (1-p)^{n-k}$ for $k = 0, 1, ..., n$	np	$np(1-p)$
Geometric(p)	$p(1-p)^{k-1}$ for $k = 1, 2, ...$	$\frac{1}{p}$	$\frac{1-p}{p^2}$
Poisson(λ)	$\frac{e^{-\lambda}\lambda^k}{k!}$ for $k = 0, 1, ...$	λ	λ
Uniform(a, b)	$\frac{1}{b-a}$ for all $x \in (a, b)$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
Gaussian(μ, σ^2)	$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ for all $x \in (-\infty, \infty)$	μ	σ^2
Exponential(λ)	$\lambda e^{-\lambda x}$ for all $x \geq 0, \lambda \geq 0$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$

LINEAR REGRESSION one-stop cheatsheet

hypothesis $h(x) = \theta^T x = \sum_{j=1}^{d} \theta_j x_j$

"regression" ≙ prediction on a continuous variable

cost: $J(\theta) = \frac{1}{2} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

To find θ that minimizes $J(\theta)$

Gradient Descent: $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$ → simultaneously performed on $j = \{0, \dots, d\}$ where d is the # of parameters.

$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} \left(\frac{1}{2} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2\right)$

check textbook for deriv.

$= \theta_j - \alpha \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)}\right) x_j^{(i)}$ ≙ WIDROW-HOFF learning rule.

≙ AKA: BATCH GRADIENT DESCENT

STOCHASTIC GRADIENT DESCENT: $\theta := \theta + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$

stochastic vs batch → stochastic updates over every training SAMPLE → may never converge
→ batch updates only after looking at entire SET

THE NORMAL EQ → minimize explicitly without the use of an iterative algorithm. Do so by $\nabla_\theta J(\theta) \stackrel{\text{set}}{=} 0$

$X \in \mathbb{R}^{n \times d} = \begin{bmatrix} -x^{(1)T}- \\ \vdots \\ -x^{(n)T}- \end{bmatrix}$ "design matrix"

$y \in \mathbb{R}^{n \times 1} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}$

$h_\theta(x^{(i)}) = x^{(i)T}\theta$

$\Rightarrow \underset{n \times d}{X} \underset{d \times 1}{\theta} - \underset{n \times 1}{y} = \begin{bmatrix} h_\theta(x^{(1)}) - y^{(1)} \\ \vdots \\ h_\theta(x^{(n)}) - y^{(n)} \end{bmatrix}$

Shown: $\frac{1}{2}(X\theta - y)^T(X\theta - y) = \frac{1}{2} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 = J(\theta)$!

$\nabla_\theta J(\theta) = \nabla_\theta \frac{1}{2}(X\theta - y)^T(X\theta - y) = X^TX\theta - X^T\vec{y} \stackrel{\text{set}}{=} 0 \rightarrow X^TX\theta = X^T\vec{y}$

derivation in textbook

closed-form soln.

Solve for θ → $\theta = (X^TX)^{-1}X^T\vec{y}$

PROBABILISTIC INTERPR. → Why is the least-squares cost function a good choice?

$y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$ Assume $\varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$

$p(\varepsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\varepsilon^{(i)2}}{2\sigma^2}\right) \Rightarrow p(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$

"probability of $y^{(i)}$ given $x^{(i)}$ when parameterized by θ"

$L(\theta) \triangleq p(\vec{y} | X; \theta) = \prod_{i=1}^{n} p(y^{(i)} | x^{(i)}; \theta)$ → $\max_\theta L(\theta)$ (we want to maximize likelihood)

$\log L(\theta) = \sum_{i=1}^{n} \log\left(p(y^{(i)} | x^{(i)}; \theta)\right) = \sum_{i=1}^{n} \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}$

$= n \log \frac{1}{\sqrt{2\pi}\sigma} - \sum_{i=1}^{n} \frac{1}{2} \frac{(y^{(i)} - \theta^T x^{(i)})^2}{\sigma^2}$ } We want to minimize this term to maximize $L(\theta)$!

$\Rightarrow \max_\theta L(\theta) \equiv \max_\theta \log(L(\theta)) \equiv \min_\theta \sum_{i=1}^{n} \frac{1}{2}(y^{(i)} - \theta^T x^{(i)})^2 \equiv J(\theta)$ FULL CIRCLE

LOCALLY-WEIGHTED LINREG (optional)

Original LinReg

① Fit θ to min $\sum_i (y^{(i)} - \theta^T x^{(i)})^2$

Locally weighted

② Fit θ to min $\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$

$w^{(i)} \triangleq \exp\left(\frac{-(x^{(i)} - x)^2}{2\tau^2}\right)$

LOOKS GAUSSIAN

- weights closer points higher!

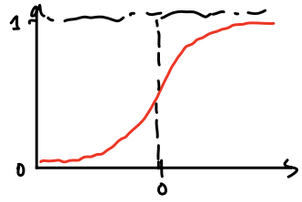
Note: this is a non-parametric algorithm. AKA no fixed set of parameters.

$O(n)$ ∴

LOG - REG one-stop

new hypothesis: $h_\theta(x) = g(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}$

Sigmoid Function 101



$g(x) = \frac{1}{1+e^{-x}}$

$g'(x) = g(x)(1-g(x))$

$0 < g(x) < 1$

Hypothesis Application:

$P(y=1 \mid x;\theta) = h_\theta(x)$

$P(y=0 \mid x;\theta) = 1 - h_\theta(x)$

More compactly:

$P(y \mid x;\theta) = h_\theta(x)^y (1-h_\theta(x))^{1-y}$

$L(\theta) = p(\vec{y} \mid X;\theta) = \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)};\theta) = \prod_{i=1}^{n} h_\theta(x^{(i)})^{y^{(i)}} (1-h_\theta(x^{(i)}))^{1-y^{(i)}}$

$\log L(\theta) = \sum_{i=1}^{n} y^{(i)} \log h_\theta(x^{(i)}) + (1-y^{(i)}) \log(1-h_\theta(x^{(i)}))$ ← how did we get here?

GRADIENT ASCENT → to find $\max_\theta \ell(\theta)$

what is $\frac{\partial}{\partial \theta_j} \ell(\theta)$? $\frac{\partial}{\partial \theta_j} \ell(\theta) = (y - h_\theta(x)) x_j$ → DERIVATION IN TEXTBOOK Bottom of Page 22.

Stochastic Gradient Ascent update: $\theta_j := \theta_j + \alpha \frac{\partial \ell}{\partial \theta} = \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$

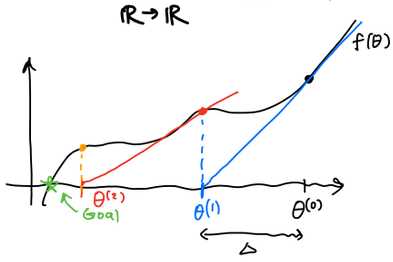
// Looks similar to Linear Regression update rule BUT h_θ is different!

NEWTON'S METHOD ("Fisher Scoring")

Alternative to iterative approach.

let $f(\theta) = \frac{\partial}{\partial \theta} \ell(\theta)$ $\mathbb{R} \to \mathbb{R}$

we want $f(\theta) = 0$



$\theta^{(1)} = \theta^{(0)} - \Delta$

$= \theta^{(0)} - \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}$

$= \theta^{(0)} - \frac{\ell'(\theta^{(0)})}{\ell''(\theta^{(0)})}$

$f'(\theta^{(0)}) = \frac{f(\theta^{(0)})}{\Delta}$

$\Delta = \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}$

set $f(\theta) = \ell'(\theta)$

General form when θ is vector-valued: $\theta := \theta - H^{-1} \nabla_\theta \ell(\theta)$

Breaking this down...

$\nabla_\theta \ell(\theta) \in \mathbb{R}^{d \times 1} = \begin{bmatrix} \frac{\partial \ell}{\partial \theta_1} \\ \vdots \\ \frac{\partial \ell}{\partial \theta_d} \end{bmatrix}$

The 'Hessian' Operator $H \in \mathbb{R}^{d \times d} = \begin{bmatrix} \frac{\partial^2}{\partial \theta_1 \theta_1} & \cdots \\ \vdots & \ddots \\ ! & \frac{\partial^2}{\partial \theta_i \theta_j} \end{bmatrix} \iff H_{ij} = \frac{\partial^2}{\partial \theta_i \partial \theta_j}$ values in Hessian

Newton's Method is typically faster than gradient ascent but is more computationally demanding given the need for an inverse Hessian (H^{-1})

GENERALIZATION & REGULATION → we ultimately care about TEST error, not TRAINING ERROR

overfitting: training error → 0, test error ↑

Underfitting: training E ↑, test error ↑

Bias & Variance

Bias ≜ test error even if we were to introduce an infinitely large training set

variance ≜ how well a model GENERALIZES.

Thought experiment for bias/variance

say regression problem comes from an underlying quadratic distribution.

- Linear fit causes high bias, no matter how many training samples you add, linear model can never fit quadratic well.
- Polynomial d-5 fit causes high variance: all quadratic polynomials ∈ d-5 polynomials, so it will fit well on most particular sets but may be off on others.

Mathematical Decomposition of Bias/Variance (for regression)

$h^*(x^{(i)})$ is an underlying distribution

- Draw a training set $S = \{x^{(i)}, y^{(i)}\}_{i=1}^n$ where $y^{(i)} = h^*(x^{(i)}) + \varepsilon^{(i)}$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$
- train a model $\hat{h}_S$ using this training set S
- Draw a test SAMPLE (x,y) and measure the Mean-Square-Error (MSE) of the trained model $\hat{h}_S(x)$. $y = h^*(x) + \varepsilon$

$$MSE(x) = E_{S,\varepsilon}[(y - \hat{h}_S(x))^2]$$

notice how this expectation is both wrt the noise ε AND the training set we took!

The Math... 😵

$$E_{S,\varepsilon}[(y - \hat{h}_S(x))^2] = E[(\varepsilon + (h^*(x) - \hat{h}_S(s)))^2]$$ // substitute $y = \varepsilon + h^*(x)$

$$= E[\varepsilon^2] + E[(h^*(s) - \hat{h}_S(s))^2]$$ // $E[(A+B)^2] = E[A^2] + E[B^2]$ for A = const.

notice this kinda looks like a variance term.

Auxiliary: let $E[\hat{h}_S(x)] = \bar{h}(x)$ which is independent of S!

intuitively $\bar{h}(x)$ is the average model you would get across an infinite # of S

$$= \underbrace{\sigma^2}_{\text{unavoidable}} + \underbrace{(h^*(x) - \bar{h}(x))^2}_{\triangleq \text{bias}^2} + \underbrace{E[(\bar{h}(x) - h_S(x))]}_{\triangleq \text{variance}}$$

Regularization

$J_\lambda(\theta) = J(\theta) + \lambda R(\theta)$ where $R(\theta)$ is a regularization term designed to "constrain the complexity" of θ.

Common regularization functions

ℓ_1 & ℓ_2 norm — $\|\theta\|_1$ & $\frac{1}{2}\|\theta\|_2$ respectively

Hold-out Cross-Validation

1. Take original data and partition into 70/30 training/validation split. (S_{train}/S_{cv})
2. Train some M_i models on the 70% train
3. Select and output $h_i(x)$ that has min $\hat{\varepsilon}_{S_{cv}}(h_i)$

$\hat{\varepsilon}_{S_{cv}}(h_i)$ ≜ average error of h on S_{cv}

K-fold CV

Artificially generates Kx the training data

leads to Kx the runtime.

S_1	S_2	S_3
Test	Train	Train
Train	Test	Train
Train	Train	Test.

$h_{ij}(x)$ ≜ hyp. of model M_i when using S_j as trainer → Select model M_i whose average error across using j as trainer is lowest.

Leave-one-out

A subset of k-fold for VERY small parent training sets S where $K = \underline{1}$ SAMPLE

K-Means & EXP. MAXIM. (EM)

K-means | Dataset $\{x^{(1)}, \ldots, x^{(n)}\}$

Algorithm: initialize clusters centroids $\mu_1, \ldots, \mu_k$ randomly

Repeat until convergence

① For each cluster $c^{(i)} = \arg\min_j \|x^{(i)} - \mu_j\|^2$ → assign each point $x^{(i)}$ a cluster based on L_2 norm from current centroids

② For each centroid $\mu_j = \frac{\sum_{i=1}^{n} 1\{c^{(i)} = j\}\, x^{(i)}}{\sum_{i=1}^{n} 1\{c^{(i)} = j\}}$ → re-define centroids as average of points $x^{(i)}$ assigned to that cluster

Distortion function $J(c, \mu) = \sum_{i=1}^{n} \|x^{(i)} - \mu_{c^{(i)}}\|^2$ → measure of "goodness" of current cluster centroids

by nature of ① & ②, J is minimized through K-means alg.

Mixture of Gaussians & EM | we have $\{x^{(1)}, \ldots, x^{(n)}\}$ observed vars & $\{z^{(1)}, \ldots, z^{(k)}\}$ hidden vars.

The hidden variables denote a multinomial distr. of k values where each k points to a separate Gaussian distribution

$z^{(i)} \sim \text{Multinom}(\phi)$ & $x^{(i)} | z^{(i)} = j \sim \mathcal{N}(\mu_j, \Sigma_j)$

We see the entire MoG model is parameterized by (ϕ, μ, Σ)

Likelihood $\ell(\phi, \mu, \Sigma) = \sum_{i=1}^{n} \log p(x^{(i)}; \phi, \mu, \Sigma)$

$= \sum_{i=1}^{n} \log \sum_{z^{(i)}=1}^{k} p(x^{(i)} | z^{(i)}; \phi, \mu, \Sigma)\, p(z^{(i)}; \phi)$

The problem term! Keep in mind $z^{(i)}$ is hidden

If $z^{(i)}$ wasn't hidden:

$\ell(\phi, \mu, \Sigma) = \sum_{i=1}^{n} \log p(x^{(i)} | z^{(i)}; \phi, \mu, \Sigma) + \log p(z^{(i)}; \phi)$

↳ $\arg\max_{\phi, \mu, \Sigma} \ell(\phi, \mu, \Sigma) \Rightarrow \phi_j = \frac{1}{n} \sum_{i=1}^{n} 1\{z^{(i)} = j\}$

$\mu_j = \frac{\sum_{i=1}^{n} 1\{z^{(i)} = j\}\, x^{(i)}}{\sum_{i=1}^{n} 1\{z^{(i)} = j\}}$ $\quad \Sigma_j = \frac{\sum_{i=1}^{n} 1\{z^{(i)} = j\}\, (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^{n} 1\{z^{(i)} = j\}}$

But $z^{(i)}$ is hidden ... Hence we follow the Exp - Max algorithm:

KERNELS → turn a nonlinear fit into a linear-model learning algorithm.

Cubic example: $\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix} \in \mathbb{R}^4$ where ϕ maps ATTRIBUTE $x \rightarrow$ FEATURES $1, x, x^2, x^3$

"feature map" $\mathbb{R}^p \rightarrow \mathbb{R}^d$ ($d=4$, $p=1$) in example

LEAST-SQUARES w/ features $\phi(x)$

$\theta := \theta + \alpha (y^{(i)} - \theta^T \phi(x^{(i)})) \phi(x^{(i)})$ // stochastic grad descent where $\theta \in \mathbb{R}^d$

Kernels

for $\phi(x^{(i)}) \in \mathbb{R}^*$ where $*$ is some crazy high # (Ex: $O(d^3)$ for $x^{(i)} \in \mathbb{R}^d$)

we don't want to iterate over $*$ using gradient descent

initialize $\theta = \vec{0} \in \mathbb{R}^*$, realize θ is a linear combination of $\phi(x^{(i)})$

$$\vec{\theta} = 0 = \sum_{i=1}^n \beta_i \phi(x^{(i)}) \rightarrow \theta := \theta + \alpha \sum_{i=1}^n (y^{(i)} - \theta^T \phi(x^{(i)})) \phi(x^{(i)})$$

$$= \sum_{i=1}^n \beta_i \phi(x^{(i)}) + \alpha \sum_{i=1}^n (y^{(i)} - \theta^T \phi(x^{(i)})) \phi(x^{(i)})$$

$$= \sum_{i=1}^n \left(\beta_i + \alpha (y^{(i)} - \theta^T \phi(x^{(i)}))\right) \phi(x^{(i)})$$

new β_i !!!

Thus we see even the updated θ value is a linear combo of $\phi(x^{(i)})$

$$\beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^n \beta_i \phi(x^{(i)})^T \phi(x^{(i)})\right)$$ // generalized β update rule.

This is your KERNAL!

Focus on this dot product: rewrite it as $\langle \phi(x^{(i)}), \phi(x^{(i)}) \rangle$, does this still not take $O(*)$ time complexity? so it's still costly to have to compute at each iteration.

Answer: ① $\langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$ can be precomputed for all i, j

② For certain definitions of $\phi(x^{(i)})$, computing $\langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$ is EASIER than $O(*)$.

Rationale ① $\langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$ is clearly static & only needs to be computed ONCE

② // Particular example in Notes pg 53.

SUMMARY!

$K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle$ → 'kernel' associated with feature map ϕ that takes $\mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$

The Algorithm:

① Compute all $K(x^{(i)}, x^{(j)})$ for all $i, j \in \{1, \ldots, n\}$

② Set $\beta = 0$ (initialize parameters)

③ Loop β update rule

$$\vec{\beta} := \vec{\beta} + \alpha (\vec{y} - K\vec{\beta})$$

$K \in \mathbb{R}^{n \times n}$

$K_{ij} = \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$

THE MUSIC: ♫♫

All relevant info on $\phi(x) \in \mathbb{R}^*$ is captured in $K \in \mathbb{R}^{n \times n}$

Reduced model fitting complexity from $O(n*) \rightarrow O(n)$!

Properties of Kernals

① $K_{ij} = K_{ji}$ → K is symmetric (dot products are commutative)

② $z^T K z \geq 0$ → K is PSD (proof on pg 57 of notes)

Generalized Learning models (GLMs)

The exponential family ≜ all distributions that can be massaged to fit the form...

$p(y;\eta) = b(y)\exp(\eta^T T(y) - a(\eta))$

- η: natural parameter (typically $\eta = f(\theta^T x)$)
- $T(y)$: sufficient statistic (typically $T(y) = y$)
- $a(\eta)$: log-partition function ($e^{-a(\eta)}$ essentially normalizes s.t. $\int p(y;\eta) = 1$.)

GLMs

Bernoulli's pg 27.

$p(y;\phi) = \phi^y(1-\phi)^{1-y}$

$= \exp(\log(\frac{\phi}{1-\phi})y + \log(1-\phi))$

$\eta = \log(\phi/1-\phi)$ $T(y) = y$

$\phi = 1/1+e^{-\eta}$

$a(\eta) = -\log(1-\phi) = \log(1+e^{\eta})$

$b(y) = 1$

Gaussian pg 28

$p(y;\mu) = \frac{1}{\sqrt{2\pi}}\exp(-\frac{1}{2}(y-\mu)^2)$

$= \frac{1}{\sqrt{2\pi}}\exp(-\frac{1}{2}y^2)\exp(\mu y - \frac{1}{2}\mu^2)$

$\eta = \mu$ $T(y) = y$

$a(\eta) = \frac{1}{2}\mu^2 = \frac{1}{2}\eta^2$

$b(y) = \frac{1}{\sqrt{2\pi}}\exp(-\frac{1}{2}y^2)$

Poisson (HW 1)

$p(y;\lambda) = e^{-\lambda}\lambda^y / y!$

$= \frac{1}{y!}\exp(y\log\lambda - \lambda)$

$b(y) = \frac{1}{y!}$ $\eta = \log\lambda$

$T(y) = y$ $a(\eta) = e^{\eta}$

canonical response function

$g(\eta) = E[y;\eta] = \lambda = e^{\eta}$

Also exists for: Multinomial, Gamma, Exponential, etc...

Constructing GLMs

3 key starting assumptions

① $(y|x;\theta) \sim$ Exponential Family (η)

② we want $h(x) = E[y|x]$

③ $\eta = \theta^T x$ (nat. param & inputs x are linearly related)

(this is how $h(x) = \theta^T x$ came to be)

Ordinary Least Squares $(y|x) \sim \mathcal{N}(\mu, \sigma^2)$

$h(x) = E[y|x;\theta] = \mu = \eta = \theta^T x$

by Assmp. ② — by def of Gaussian — shown — by Assmp. ③

Logistic Regression $y|x;\theta \sim$ Bern (ϕ)

$h_\theta(x) = E[y|x;\theta] = \phi = 1/1+e^{-\eta} = \frac{1}{1+e^{-\theta^T x}}$

② — def. of Bern. — shown — ③

Softmax Regression → GLM for a multinomial distribution.

Multinom ($\phi_1, \dots, \phi_k$) where $\phi_i \triangleq p(y=i)$. Notice only $k-1$ indp. params. since $\phi_k = 1 - \sum_{i=1}^{k-1}\phi_i$

$T(y) \in \mathbb{R}^{k-1\times 1}$ $T(1) \triangleq \begin{bmatrix}1\\0\\0\\\vdots\end{bmatrix}$ $T(2) \triangleq \begin{bmatrix}0\\1\\0\\\vdots\end{bmatrix}$ ... $T(k-1) \triangleq \begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix}$ $T(k) \triangleq \begin{bmatrix}0\\\vdots\\0\end{bmatrix}$

important to understand: $T(y)_i = \mathbb{1}\{y=i\}$, $E[T(y)_i] = p(y=i) = \phi_i$

$\Rightarrow p(y;\phi) = \phi_1^{\mathbb{1}\{y=1\}}\phi_2^{\mathbb{1}\{y=2\}}\dots\phi_k^{1-\sum_{i=1}^{k-1}\mathbb{1}\{y=i\}} = \phi_1^{T(y)_1}\phi_2^{T(y)_2}\dots\phi_k^{1-\sum_{i=1}^{k-1}T(y)_i}$

pg 32 $\in \mathbb{R}^{k-1\times 1}$

$p(y;\phi) = b(y)\exp(\eta^T T(y) - a(\eta))$ with $b(y) = 1$, $a(\eta) = -\log(\phi_k)$

$\eta = \begin{bmatrix}\log(\phi_1/\phi_k)\\\log(\phi_2/\phi_k)\\\vdots\\\log(\phi_{k-1}/\phi_k)\end{bmatrix} \Leftrightarrow \eta_i = \log(\frac{\phi_i}{\phi_k})$

$\Rightarrow \phi_i = \frac{e^{\eta_i}}{\sum_{j=1}^{k} e^{\eta_j}}$ $\Rightarrow p(y=i|x;\theta) = \phi_i = \frac{e^{\eta_i}}{\sum_{j=1}^{k} e^{\eta_j}} = \frac{e^{\theta_i^T x}}{\sum_{j=1}^{k} e^{\theta_i^T x}}$

softmax funct. — Softmax regression

$h_\theta(x) = \begin{bmatrix}\phi_1\\\vdots\\\phi_{k-1}\end{bmatrix}$ → which can be written in terms of the softmax regression equation

GENERATIVE LEARNING

Instead of $p(y|x)$ (discriminative models) learn $p(x|y)$ (generative models)

Map of $p(x|y) \leftrightarrow p(y|x)$ is BAYE'S RULE

$$p(y|x) = \frac{p(x|y)\,p(y)}{p(x)} \rightsquigarrow \arg\max_y p(y|x) = \arg\max_y p(x|y)\,p(y)$$

GDA For math context on multivariate Gaussians → see Probability One-stop note

Consider discrete classification problem of y on continuous variables x

$y \sim \text{Bern}(\phi)$ $\quad p(y) = \phi^y (1-\phi)^{(1-y)}$

$(x|y=0) \sim \mathcal{N}(\mu_0, \Sigma)$ $\quad p(x|y=0) = \frac{1}{2\pi^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_0)^T\Sigma^{-1}(x-\mu_0)\right)$

$(x|y=1) \sim \mathcal{N}(\mu_1, \Sigma)$ $\quad p(x|y=1) = \frac{1}{2\pi^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_1)^T\Sigma^{-1}(x-\mu_1)\right)$

(Derived painfully in HW 2.)

$$\phi = \frac{1}{n}\sum_{i=1}^n \mathbb{1}\{y^{(i)}=1\}$$

$$\mu_{0/1} = \frac{\sum_{i=1}^n \mathbb{1}\{y^{(i)}=0/1\}\,x^{(i)}}{\sum_{i=1}^n \mathbb{1}\{y^{(i)}=0/1\}}$$

$$\Sigma = \frac{1}{n}\sum_{i=1}^n (x^{(i)}-\mu_{y^{(i)}})(x^{(i)}-\mu_{y^{(i)}})^T$$

NAIVE BAYES Important yet strong Naive Bayes Assumption: $p(x_i|y)$ are conditionally independent on y

Let $x_j \sim \text{Bern}(\phi)$ $\quad x = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \end{bmatrix}$

Naive Bayes is parameterized by

$$\phi_{j|y=1} = \frac{\sum_{i=1}^n \mathbb{1}\{x_j^{(i)}=1 \wedge y^{(i)}=1\}}{\sum_{i=1}^n \mathbb{1}\{y^{(i)}=1\}} \qquad \phi_{j|y=0} = \frac{\sum_{i=1}^n \mathbb{1}\{x_j^{(i)}=1 \wedge y^{(i)}=0\}}{\sum_{i=1}^n \mathbb{1}\{y^{(i)}=0\}}$$

all of these are essentially just fractions

$$\phi_y = \frac{\sum_{i=1}^n \mathbb{1}\{y^{(i)}=1\}}{n}$$

The gist of Naive Bayes

$$p(y=1|x) = \frac{\left(\prod_{j=1}^d p(x_j|y=1)\right)p(y=1)}{\left(\prod_{j=1}^d p(x_j|y=1)\right)p(y=1) + \left(\prod_{j=1}^d p(x_j|y=0)\right)p(y=0)} = \frac{(\phi_{j|y=1})(\phi_y)}{(\phi_{j|y=1})(\phi_y) + (\phi_{j|y=0})(1-\phi_y)}$$

LAPLACE SMOOTHING if there is a word in your dictionary that never pops up in your set ... you get $p(y=1|x) = \frac{0}{0+0} = \frac{0}{0}$ ∴

Simply add something to $\phi_{j|y=1}$ or $\phi_{j|y=0}$ while still keeping $\sum_j \phi_{j|y=0/1} = 1$

$$\phi_{j|y=1} = \frac{1 + \sum_{i=1}^n \mathbb{1}\{x_j^{(i)}=1 \wedge y^{(i)}=1\}}{2 + \sum_{i=1}^n \mathbb{1}\{y^{(i)}=1\}}$$

Always 1

2 because $x_j \sim$ Bern which only has 2 outcomes

DECISION TREES

Regions $\{R_1, \ldots, R_k\}$ that partition the input domain
predictions $\{w_1, \ldots, w_k\} \rightarrow f(x) = \sum_{j=1}^{k} w_j \mathbb{1}\{x \in R_j\}$

common regression prediction.

$$w_j = \frac{\sum_{i=1}^{n} y^{(i)} \mathbb{1}\{x^{(i)} \in R_j\}}{\sum_{i=1}^{n} \mathbb{1}\{x^{(i)} \in R_j\}}$$

common classification prediction:
w_j = most common class in R_j

Model Fitting → NP-complete problem

general idea: $\mathcal{L}(f) = \sum_{j=1}^{k} \sum_{x^{(i)} \in R_j} \ell(y^{(i)}, w_j)$ → Loss function

Find $\min_f \mathcal{L}(f)$

for every region, for every sample in region...

Greedy Algorithm

① Start with a SINGLE NODE that contains all training samples.

② Choose a split s.t. $j, \theta = \arg\max_{j,\theta} G(j,\theta)$
- where j ≙ feature to split $\in \{1, \ldots d\}$
- θ ≙ value of feature j to split at $\in R$
- $G(j,\theta)$ ≙ gain function (utility to split)

③ Recurse on split nodes

④ BASE CASE FOR STOP RECURSE:
- error = 0
- no more features to split on

Caveat: if θ is continuous, the valid points to split are MIDPOINTS of observed samples.

in n sorted continuous samples $\{x^{(1)}, \ldots, x^{(n)}\}$,
$\theta \in \left\{ \frac{x^{(2)}+x^{(1)}}{2}, \ldots, \frac{x^{(n)}+x^{(n-1)}}{2} \right\} \in \mathbb{R}^{n-1}$

Define G

$$G(j,\theta) \triangleq C(S) - \left[\frac{|L|}{|S|} C(L) + \frac{|R|}{|S|} C(R) \right]$$

(a very intuitive gain function)

notice!: $\arg\max_{j,\theta} G(j,\theta) \equiv \arg\min_{j,\theta} C(\text{after split})$

Define C The cost function can take many values given problem type.

→ Regression: $C(S) = \frac{1}{|S|} \sum_{x^{(i)} \in R} (y^{(i)} - w)^2$

→ Classification: $C(S) = \sum_{c \in C} -P_c \log P_c$

where C is a class space & P_c ≙ fraction of datapoints in S with class c.

PREVENTING TREE OVERFITTING

① limiting # of leaf nodes or tree depth
② no split is # data in node is too small
③ Pruning: sort of like a greedy-removal alg.

Scoring a Tree $C(T) = Err(T) + \lambda L(T)$

where T ≙ a tree
$C(T)$ ≙ cost of said tree
$Err(T)$ ≙ error of tree
$L(T)$ ≙ # leaves in tree
λ ≙ regularization constant!
- → $\lambda = \infty$ → Root (no splits!)
- → $\lambda = 0$ → maximal tree!

ENSEMBLE LEARNING → idea: letting a couple models work together
- reduces variance with no sig. impact on bias

Let $F_1, \dots, F_m$ be a set of models

$f(x) = \sum_{i=1}^{m} \beta_i F_i(x)$ where $\beta_i \triangleq$ weight for each model's prediction (typically $= \frac{1}{m}$)

basically ensemble learning.

Bagging: Generate a new training set. for each model
- original training set size n —sample n data with replacement→ new training data set size n.
- Note: there is a $(1-e^{-1}) \approx 0.63$ proportion of original points that appear!

Random Forest: a variant of bagging for TREES only.
- at each splitting node, run splitting algorithm on a subset of data!

Boosting: idea is to combine WEAK learners to make STRONG learners

$$\beta_i, \theta_i = \arg\min_{\beta,\theta} \sum_{j=1}^{n} \ell\left(y^{(j)}, \left(f_{i-1}(x^{(j)}) + \beta F_i(x^{(j)}; \theta)\right)\right)$$

// forward-stagewise additive modelling with F_i as the current model in-design

$f_i(x) := f_{i-1}(x) + \beta_i F_i(x)$ // model updating

Let $\gamma \in (0, \frac{1}{2})$ $\varepsilon \in (0,1)$ $\delta \in (0,1)$ $m \triangleq$ # iid samples learned from.

Strong learner $\triangleq$ $\forall(\varepsilon, \delta)\ \exists(m)$ s.t. $P(\text{Accuracy}(f) > 1-\varepsilon) \geq 1-\delta$

(I'm confident with enough m, I can reach near 100% accur.)

Weak learner $\triangleq$ $\forall(\delta)\ \exists(m)$ s.t. $P(\text{Accuracy}(f) > \frac{1}{2}+\gamma) \geq 1-\delta$

(I'm confident with enough m, I can be at least 50% correct)